



HTTP

Integração de Sistemas - Aula #04

Prof. Dr. Carlos Melo

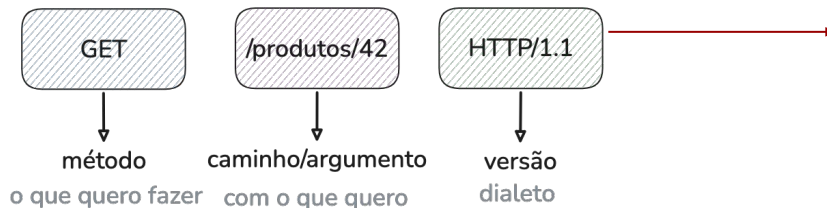
 carlos.melo@upe.br



Continuando...

- Desde a aula passada estamos usando o HTTP, inclusive vide RPC
 - `servico = xmlrpc.client.ServerProxy("http://localhost:8080")`
- Existem dezenas de protocolos de integração:
 - IIOP, DCOM, IBM MQ, XML-RPC, etc.
- Mas todos eles usam HTTP como espinha dorsal

Requisição



```
carlos@m1:~  
python3 (Python*)  
  
> python3 -m http.server 8000  
Serving HTTP on :: port 8000 (http://[::]:8000/) ...  
:::1 - - [29/Aug/2026 08:06:00] "GET / HTTP/1.1" 200 -  
[ ]  
  
~ (-zsh)  
Last login: Sat Aug 29 07:43:10 on ttys000  
  
>  
>  
>  
>  
  
> nc localhost 8000  
GET / HTTP/1.1  
  
HTTP/1.0 200 OK  
Server: SimpleHTTP/0.6 Python/3.13.3  
Date: Sat, 29 Aug 2026 11:06:00 GMT  
Content-type: text/html; charset=utf-8  
Content-Length: 3914
```



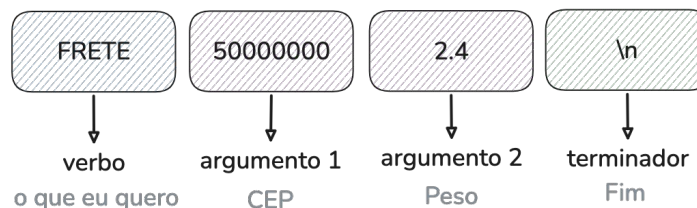
Nada que vocês não já tenham feito antes

- O HTTP adiciona:
 - Versão do protocolo
 - Cabeçalhos livres
 - Corpo opcional
 - Códigos de resposta padronizados
 - E é um idioma global

Requisição

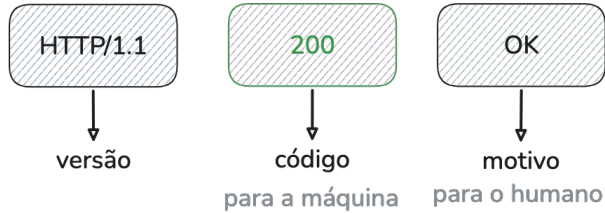


Pedido



A resposta

Resposta



- Corpo com tamanho em bytes
- Formato do conteúdo

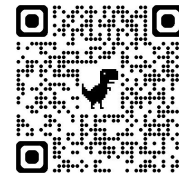
```
carlos@m1:~  
python3 (Python*)  
~  
> python3 -m http.server 8000  
Serving HTTP on :: port 8000 (http://[::]:8000/) ...  
:::1 - - [29/Aug/2026 08:06:00] "GET / HTTP/1.1" 200 -  
[  
~  
Last login: Sat Aug 29 07:43:10 on ttys000  
~  
>  
~  
>  
~  
> nc localhost 8000  
GET / HTTP/1.1  
  
HTTP/1.0 200 OK  
Server: SimpleHTTP/0.6 Python/3.13.3  
Date: Sat, 29 Aug 2026 11:06:00 GMT  
Content-type: text/html; charset=utf-8  
Content-Length: 3914
```

Enquadramento no HTTP

- O HTTP usa delimitador para cabeçalho e tamanho para o corpo

```
carlos@m1:~  
python3 (Python*)  
~  
> python3 -m http.server 8000  
Serving HTTP on :: port 8000 (http://[::]:8000/) ...  
:::1 - - [29/Aug/2026 08:06:00] "GET / HTTP/1.1" 200 -  
~  
~ (-zsh)  
Last login: Sat Aug 29 07:43:10 on ttys000  
~  
>  
~  
>  
~  
> nc localhost 8000  
GET / HTTP/1.1  
  
HTTP/1.0 200 OK  
Server: SimpleHTTP/0.6 Python/3.13.3  
Date: Sat, 29 Aug 2026 11:06:00 GMT  
Content-type: text/html; charset=utf-8  
Content-Length: 3914
```

Métodos



	Significado	Muda algo no servidor?
GET	Me dê	Não
POST	Tome isso e faça algo	Sim
PUT	Guarde isso neste lugar	Sim
PATCH	Altere só esta parte	Sim
DELETE	Remova	Sim

Status

1xx segura aí

informativo

2xx deu certo

200 OK - 201 Criado - 204 Sem conteúdo

3xx procure em outro lugar

301 mudou de vez - 304 use o cache

4xx o erro foi seu

400 pedido mal formatado - 401 quem é tu?
- 404 Não encontrado

5xx o erro foi meu

500 quebrei - 503 estou fora do ar



<https://http.dog/>

carlos.melo@upe.br



Tudo que não é conteúdo é cabeçalho

- Quem dá sentido ao corpo é justamente o cabeçalho
- GET normalmente não têm corpo
- Toda resposta com corpo deveria dizer o tipo



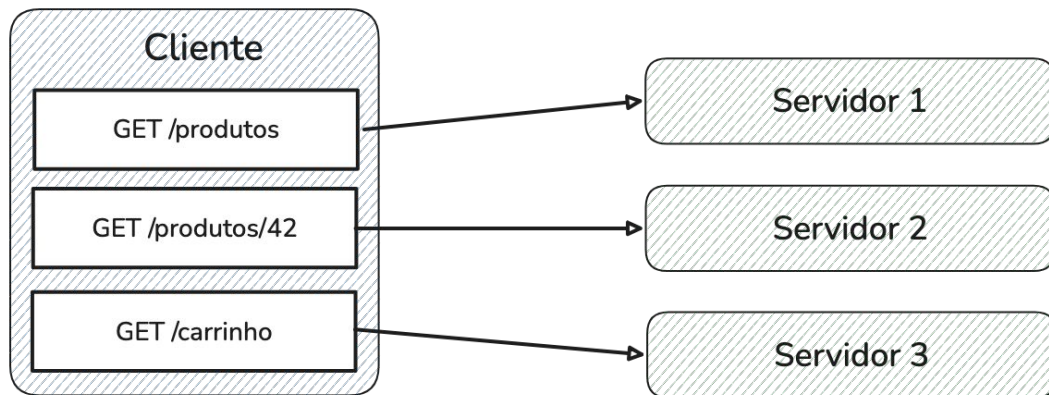
```
1 Host: api.loja.com.br           para qual site, neste endereço
2 Content-Type: application/json  como ler o corpo
3 Content-Length: 31              quantos bytes ele tem
4 Authorization: Bearer abc123    quem está falando
5 Accept: application/json        o que eu sei receber
6 User-Agent: curl/8.4.0          quem é o programa
```




Sem estado

O HTTP não guarda nada entre uma requisição e outra.

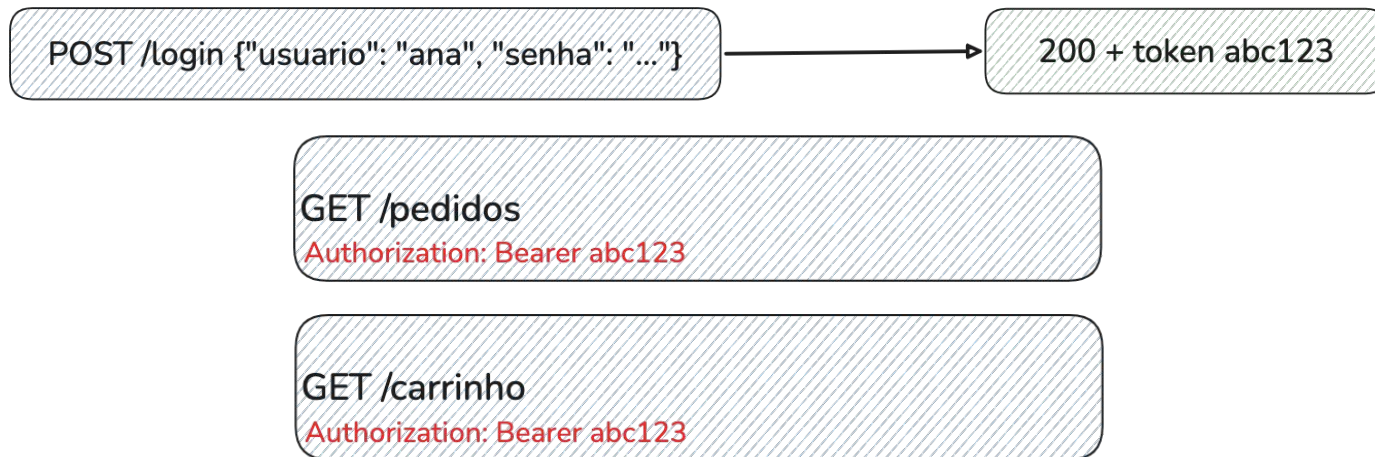
- Cada requisição carrega tudo o que o servidor precisa saber
- Como nenhum deles guarda nada, a máquina atende qualquer pedido
- É por isso que dá para acrescentar uma máquina sem avisar ninguém





Então como a página sabe quem sou eu?

- O servidor continua sem lembrar. Quem carrega a identidade é o cliente
- Cookie e token transformam estado em mais um cabeçalho e eu posso ter quantos cabeçalhos quiser (a requisição só vai acabar na linha em branco).





HTTPS

- É o mesmo HTTP, dentro de um túnel cifrado (Transport Layer Security - TLS)
 - Métodos, status, cabeçalhos e corpo, tudo a mesma coisa
- As únicas mudanças é que ninguém no caminho lê ou altera o conteúdo da requisição ou da resposta e que roda na porta 443 ao invés da 80
- O mesmo cliente vai funcionar para HTTP e HTTPS
 - O navegador é o principal cliente com uma interface gráfica para usar ambos, mas não é o único cliente

curl



```
> curl https://viacep.com.br/ws/50720001/json/
```

```
{
  "cep": "50720-001",
  "logradouro": "Rua Benfica",
  "complemento": "",
  "unidade": "",
  "bairro": "Madalena",
  "localidade": "Recife",
  "uf": "PE",
  "estado": "Pernambuco",
  "regiao": "Nordeste",
  "ibge": "2611606",
  "gia": "",
  "ddd": "81",
  "siafi": "2531"
}
```



```
1 curl https://viacep.com.br/ws/50720001/json/
2
3 curl -i ... # mostra a linha de status e os cabeçalhos
4 curl -v ... # mostra a conversa inteira
5 curl -X POST -H "Content-Type: application/json" \
6     -d '{"nome":"Ana"}' https://httpbin.org/post
```

```
> curl -v https://viacep.com.br/ws/50720001/json/
```

```
* Host viacep.com.br:443 was resolved.
* IPv6: (none)
* IPv4: 165.227.126.241
*   Trying 165.227.126.241:443...
* Connected to viacep.com.br (165.227.126.241) port 443
* ALPN: curl offers h2,http/1.1
* (304) (OUT), TLS handshake, Client hello (1):
* CAfile: /etc/ssl/cert.pem
* CApath: none
* (304) (IN), TLS handshake, Server hello (2):
* (304) (IN), TLS handshake, Unknown (8):
* (304) (IN), TLS handshake, Certificate (11):
* (304) (IN), TLS handshake, CERT verify (15):
* (304) (IN), TLS handshake, Finished (20):
* (304) (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / AEAD-AES256-GCM-SHA384 / [blank] / UNDEF
* ALPN: server accepted http/1.1
```

Exercício



Use o ViaCEP que é público e não pede cadastro através do curl

1. Identifique status, Content-Type e Content-Length
2. `curl -v` no mesmo endereço e ache a linha em branco entre cabeçalhos e corpo
3. Peça um CEP mal formado, com 9 dígitos. Que status voltou?
4. Peça um CEP bem formado mas inexistente, 99999999. Que status voltou, e o que veio no corpo?
5. Compare 3 e 4.
6. `curl -X POST -d '{"a":1}' -H "Content-Type: application/json" https://httpbin.org/post`

APIs



Uma porta que um sistema abre para outros programas usarem (não para pessoas).

- Os Correios têm a base de CEPs do Brasil.
 - Você não tem acesso ao banco de dados deles, não sabe em que máquina ele roda, não sabe em que linguagem foi escrito.
 - Mas existe um endereço que você pode chamar:

```
1 GET https://viacep.com.br/ws/50720001/json/  
2  
3 { "cep": "50720-001",  
4   "logradouro": "...",  
5   "bairro": "...",  
6   "localidade": "Recife",  
7   "uf": "PE" }
```

Esse acordo é API:

- O que dá para pedir: um CEP de 8 dígitos
- Como pedir: via GET nesse formato de endereço
- O que volta: Um JSON com esses campos.



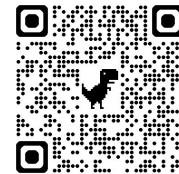
Para que serve o REST?

- Sem estilo, cada API é um vocabulário novo:
 - `criarPedido`, `listarPedidos`, `pedidoDetalhe` para cada serviço, um por um.
- Com estilo, quem sabe HTTP já sabe usar a sua API.



todo mundo entende GET, POST, 200, 404, etc
Ninguém sabe o que danado é pedido, frete, cep e cliente

REST na prática



1 GET	/pedidos	lista	200
2 POST	/pedidos	cria	201
3 GET	/pedidos/42	mostra	200 ou 404
4 PATCH	/pedidos/42	altera	200
5 DELETE	/pedidos/42	remove	204



E as ações que não são substantivos?

aprovarPedido, cancelar, reenviar

- PATCH /pedidos/42 com {"status": "aprovado"}
- ou POST /pedidos/42/aprovacoes
- ou aceite o verbo e documento: POST /pedidos/42/aprovar



O que vamos construir

Uma API de frete que consulta o ViaCEP para descobrir a UF.

- GET /fretes/50720001
 - { "uf": "PE", "valor": 18.90 }

```
1 import httpx
2 from fastapi import FastAPI, HTTPException
3
4 app = FastAPI()
5 TABELA = {"PE": 18.90, "SP": 24.50, "AM": 41.00}
6
7 @app.get("/fretes/{cep}")
8 def calcular(cep: str):
9     r = httpx.get(f"https://viacep.com.br/ws/{cep}/json/", timeout=3)
10    dados = r.json()
11    uf = dados["uf"]
12    return {"uf": uf, "valor": TABELA[uf]}
```

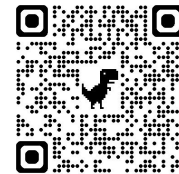
- Até funciona, porém...



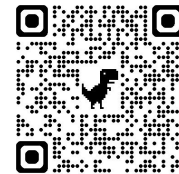
O que acontece se:

- Se o CEP estiver mal formatado o ViaCEP responde 400 e o `r.json()` estoura
- Se o CEP for inexistente o ViaCEP retorna 200 o que vai gerar um `KeyError` no `'UF'`
- UF fora da tabela com `KeyError 'RR'`
- ViaCEP demora a responder vai gerar uma exceção por timeout

Exercício



- Corrija a bucha alheia e integre o ViaCEP à nossa API.



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br